Elastic Net Regression

Contents

1. Overview 2
2. The loss function 3
3. alpha and l1_ratio 5
4. The shape of the penalty 8
5. Correlated features: the grouping effect 10
6. Elastic Net on the diabetes data 12
7. When to use which 14
8. Summary 16
9. Sources 16
10. Key terms 16

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Multicollinearity; Regularisation; Ridge regression; Lasso regression.

1. Overview

Key point: Elastic Net adds both the Ridge penalty and the Lasso penalty to the loss. One setting controls the overall strength, another the mix between the two.

Ridge and Lasso are two regularised versions of linear regression (a penalty on large coefficients is added to the loss):

- **Ridge** (L2, **L2 regularisation**, G-1029): shrinks all coefficients, never to exactly 0. Suited to data where every **feature** (G-772) (an input variable, one column of the data table) matters.
- **Lasso** (L1, **L1 regularisation**, G-1026): can set coefficients to exactly 0, removing features. Suited to data where only some features matter.

Figure 1 recalls the difference on a made-up model of house prices with two useful features (area, rooms) and two useless ones (door number, the owner's birth month). Watch the two grey terms: Ridge keeps them with small coefficients, Lasso removes them.

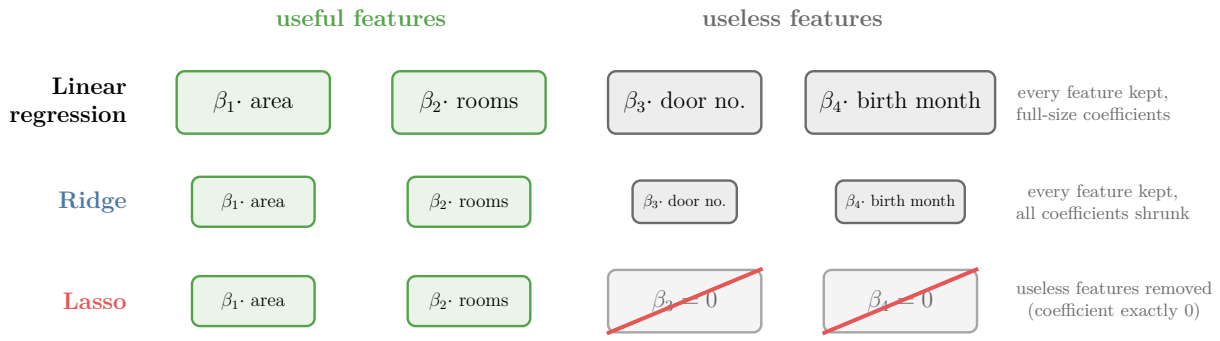


Figure 1: One model, three fits. Linear regression keeps every feature at full size. Ridge keeps every feature and shrinks all coefficients. Lasso sets the coefficients of the useless features to exactly 0, which removes them from the model. Idea after StatQuest, "Regularization Part 3: Elastic Net Regression"; the example is our own

The problem. Choosing between Ridge and Lasso needs knowledge of the features: are most of them useful, or only a few? With four features we can tell. With hundreds or thousands of features, nobody can check them one by one, so there is no way to choose in advance.

The fix. There is no need to choose. **Elastic Net regression** (G-667) adds both penalties to the loss at once, each with its own strength, and the two strengths are tuned like any

other **hyperparameter** (G-910).

2. The loss function

> **Key point:** Loss = squared error + a × (sum of squared coefficients) + b × (sum of absolute coefficients).

In plain words: start from the squared error of linear regression, add the Ridge penalty with its own strength a, and add the Lasso penalty with its own strength b.

Worked example. Three houses, two **features** (G-772): area (x_1, in hundreds of square feet) and rooms (x_2). The **target** (G-1949) y is the price in lakh rupees. A model with coefficients $\beta_1 = 2$ and $\beta_2 = 1$ predicts the price $\hat{y}$ from the area and the rooms:

$$\hat{y} = 2x_1 + 1x_2$$

Take the strengths $a = 0.5$ and $b = 0.5$.

Step 1, the squared error, one house per row:

House	x_1	x_2	Price y	Prediction $\hat{y}$	Error $y - \hat{y}$	Squared
1	3	2	9	$2 \times 3 + 1 \times 2 = 8$	1	1
2	4	3	10	$2 \times 4 + 1 \times 3 = 11$	-1	1
3	5	1	12	$2 \times 5 + 1 \times 1 = 11$	1	1
Sum						3

Step 2, the Ridge penalty: square each coefficient, then add.

$$\beta_1^2 = 2^2 = 4$$

$$\beta_2^2 = 1^2 = 1$$

$$\text{sum} = 4 + 1 = 5$$

$$a \times 5 = 0.5 \times 5 = 2.5$$

Step 3, the Lasso penalty: take each coefficient without its sign, then add.

$$|\beta_1| = 2$$

$$|\beta_2| = 1$$

$$\text{sum} = 2 + 1 = 3$$

$$b \times 3 = 0.5 \times 3 = 1.5$$

Step 4, the loss:

$$L = 3 + 2.5 + 1.5 = 7$$

Training chooses the coefficients that make this number smallest. With $a = b = 0$ the loss would be just 3, the squared error of plain linear regression.

The same calculation as a formula. The symbol $\sum_{i=1}^{n}$ means "add the term for house $i = 1$, then $i = 2$, up to $i = n$"; here $n = 3$ houses. Likewise $\sum_{j=1}^{m}$ adds over the m features; here $m = 2$. With y_i the price of house i, $\hat{y}_i$ its prediction and β_j the coefficient of feature j:

$$L = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

$$+a \sum_{j=1}^{m} \beta_j^2$$

$$+b \sum_{j=1}^{m} |\beta_j|$$

On the example, the three sums are 3, 5 and 3, so as above:

$$L = 3 + 0.5 \times 5 + 0.5 \times 3 = 7$$

a sets the strength of the Ridge part and b the strength of the Lasso part. They are separate numbers, so the two parts can be weighted differently. Two strengths give four cases:

a (Ridge)	b (Lasso)	Model
0	0	plain linear regression
above 0	0	Ridge
0	above 0	Lasso

a (Ridge)	b (Lasso)	Model
above 0	above 0	Elastic Net, a hybrid of the two

The best pair (a, b) is found by **cross-validation** (G-510) (section 6).

Figure 2 places every model on one map of the two strengths. Watch the edges: they are the models of the earlier Notes, and only the inside is new.

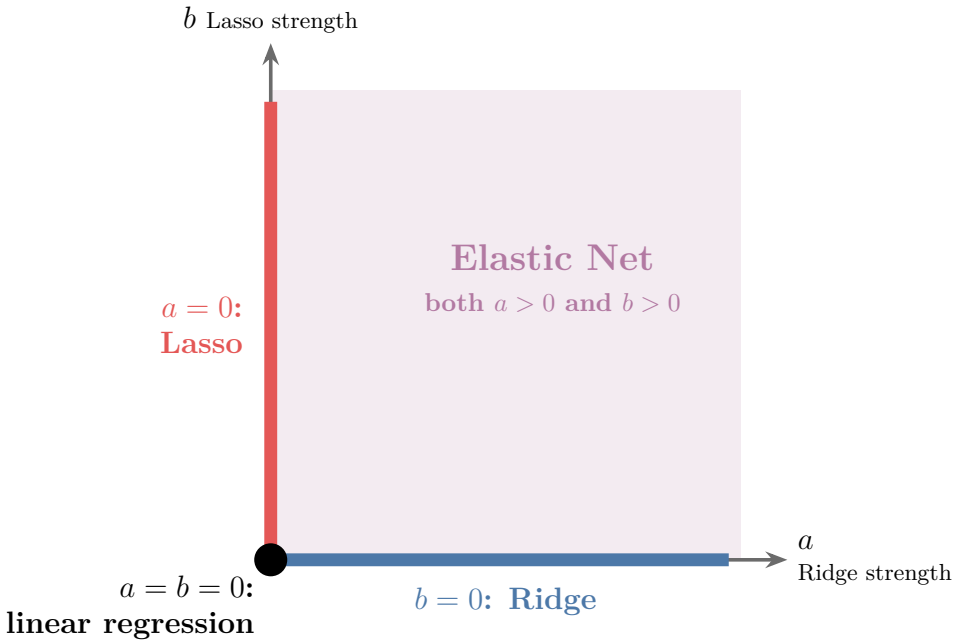


Figure 2: Each pair of strengths (a, b) is one model. The horizontal edge (b = 0) is Ridge, the vertical edge (a = 0) is Lasso, the corner (a = b = 0) is linear regression, and everything inside is Elastic Net.

3. alpha and l1_ratio

> **Key point:** scikit-learn uses alpha for the total strength and l1_ratio for the share of the Lasso (L1) part.

Instead of a and b, scikit-learn's `ElasticNet` takes two other hyperparameters:

$$\text{alpha} = a + b$$

$$r = \frac{b}{a + b}$$

Here r is the **l1_ratio** (G-1027) **hyperparameter** (G-910): a setting chosen before training, not learned from the data.

In words: `alpha` is the total penalty, and `l1_ratio` is the fraction of it that goes to the L1 (Lasso) part. The rest, $1 - r$, goes to the L2 (Ridge) part.

l1_ratio	Mix	Behaves like
0	all L2	Ridge
0.5 (default)	half and half	an even mix
0.9	90% L1, 10% L2	mostly Lasso
1	all L1	Lasso

Figure 3 draws the same map in scikit-learn's terms. Watch the four rows of the table: all have alpha 1, so they sit on one line, and l1_ratio only turns the direction from the Ridge edge to the Lasso edge.

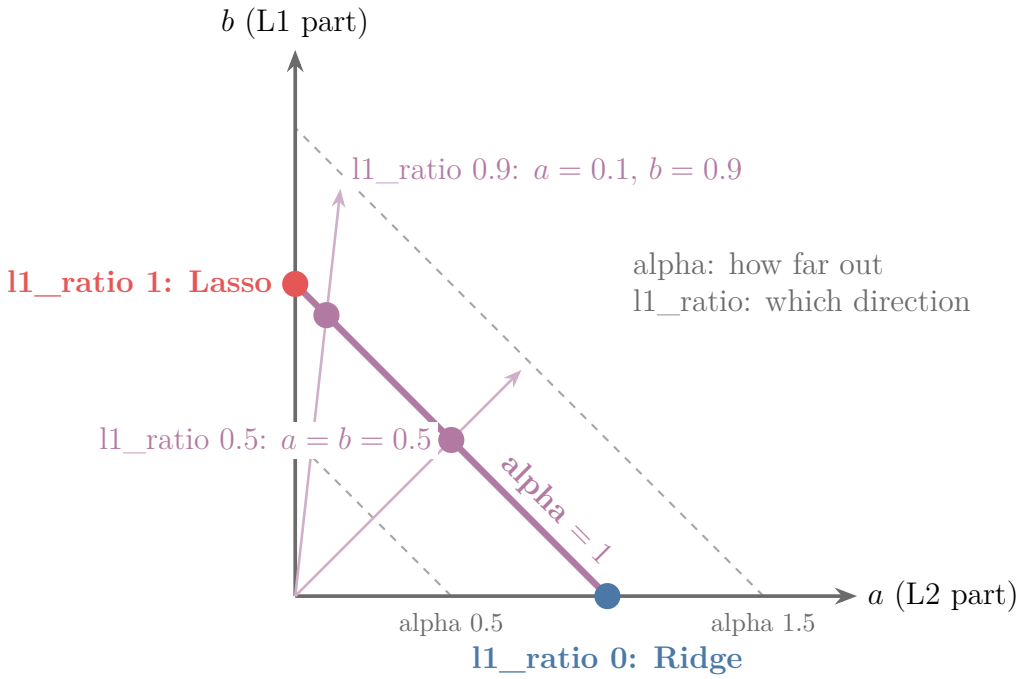


Figure 3: alpha and l1_ratio on the map of Figure 2. The dashed lines are constant alpha (a + b); the purple line is alpha 1. The four table rows sit on that line: l1_ratio 0 at (1, 0), 0.5 at (0.5, 0.5), 0.9 at (0.1, 0.9) and 1 at (0, 1).

With numbers, using the strengths of Section 2, $a = 0.5$ and $b = 0.5$:

$$\text{alpha} = a + b = 0.5 + 0.5 = 1$$

$$r = \frac{b}{a+b} = \frac{0.5}{1} = 0.5$$

Going back: alpha 1 and l1_ratio 0.5 give

$$b = \text{alpha} \times r$$

$$= 1 \times 0.5 = 0.5$$

$$a = \text{alpha} - b$$

$$= 1 - 0.5 = 0.5$$

Extra: A common slip is to read l1_ratio 0.9 as "90% Ridge". The name says what it measures: the L1 share. So 0.9 means 90% Lasso (scikit-learn docs, `ElasticNet`).

Extra: As with `Lasso`, scikit-learn scales the terms a little differently. It minimises

$$\frac{1}{2n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$+\text{alpha} \cdot r \sum_{j=1}^{m} |\beta_j|$$

$$+\frac{1}{2}\text{alpha}(1-r)\sum_{j=1}^{m}\beta_j^2$$

with r the `l1_ratio`. On the three-house example with alpha 1 and $r = 0.5$:

$$\frac{1}{2 \times 3} \times 3 = 0.5$$

$$1 \times 0.5 \times 3 = 1.5$$

$$\frac{1}{2} \times 1 \times 0.5 \times 5 = 1.25$$

$$0.5 + 1.5 + 1.25 = 3.25$$

The idea is the same; only the scale of alpha differs (scikit-learn docs, `ElasticNet`).

4. The shape of the penalty

> **Key point:** The Elastic Net penalty is between Ridge's circle and Lasso's diamond: rounded, but still with corners on the axes, so it can still set coefficients to 0.

For two coefficients, called w_1 and w_2 in Figure 4, the figure draws all the points where each penalty equals 1.

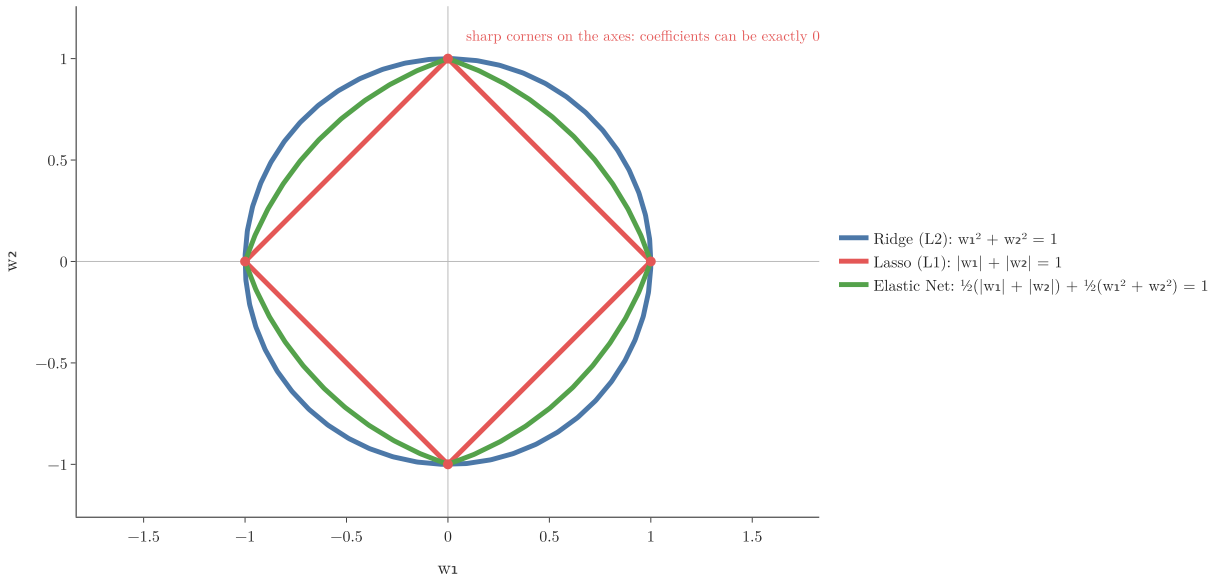


Figure 4: The penalty shapes of Ridge, Lasso and Elastic Net

- **Ridge** gives a circle: smooth everywhere.
- **Lasso** gives a diamond with sharp corners on the axes, where one coefficient is 0. Those corners are why Lasso answers often land exactly on 0 (the diamond and the circle grows the loss ellipse, a ring of equal loss around the least-squares answer, until it touches the diamond).
- **Elastic Net** is between the two. Its sides bulge outwards like the circle, but it keeps the corners, so it can still produce exact zeros (Zou and Hastie 2005, Fig. 1).

Figure 5 turns the shape from diamond into circle on real data: two diabetes **features** (G-772), bmi and s1, from the training split of feature selection, each scaled to standard deviation 1. The budget is fixed and only `l1_ratio` changes. Each shape is the **feasible region** (G-759, see the diamond and the circle): the set of coefficient pairs inside the budget (its formula is below). Every shape except the circle at `l1_ratio` 0 keeps corners on the axes. In each frame the left panel shows the shape (red diamond to blue circle) and one orange arc of the loss ellipse; watch the dot where the arc touches the shape (the least-squares answer is the black dot), and the s1 coefficient traced in green against `l1_ratio` on the right.

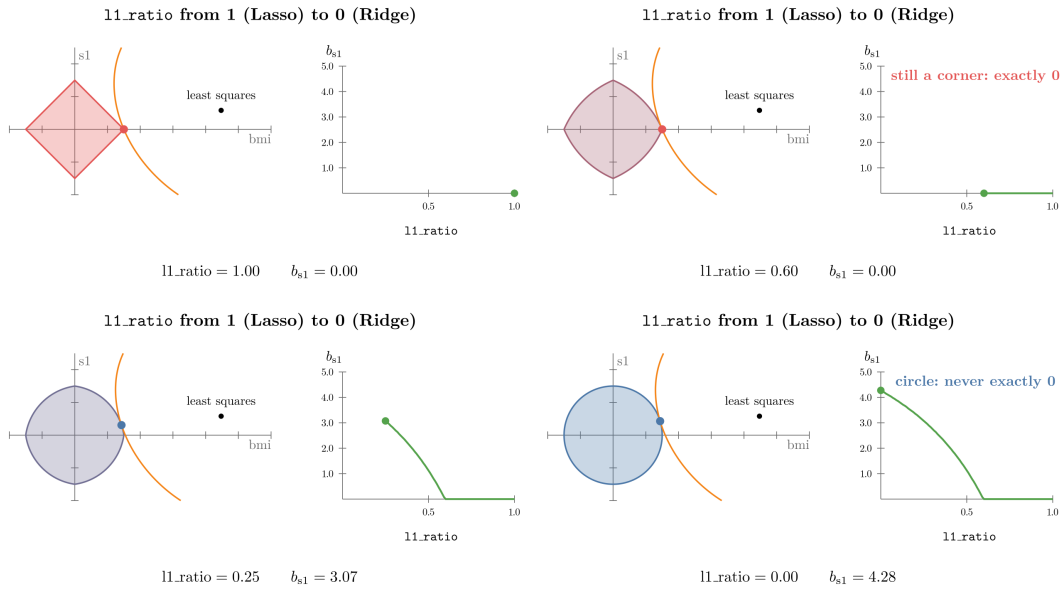


Figure 5: The feasible region morphs from Lasso's diamond (l1_ratio 1) to Ridge's circle (l1_ratio 0) on two diabetes features. The s1 coefficient, where the loss ellipse first touches the region, stays exactly 0 down to l1_ratio 0.6 and then grows to 4.28

- **l1_ratio from 1 down to 0.6:** the touch stays on the corner, so the s1 coefficient is exactly 0, as in Lasso.
- **Below 0.6:** the shape is round enough that the touch slides off the axis; at 0 (Ridge) the s1 coefficient is 4.28.

The weak feature s1 is dropped over a whole range of mixes, not only at pure Lasso.

The formula behind the shapes. Call the two coefficients β_1 (bmi) and β_2 (s1), as in Section 2, the budget 15, and the mix r (the `l1_ratio`). A pair (β_1, β_2) is inside the region when

$$r\,\frac{|\beta_1| + |\beta_2|}{15}$$

$$+(1-r)\,\frac{\beta_1^2 + \beta_2^2}{15^2} \le 1$$

Check it at the two ends, then at one point in between:

r	Formula	Shape
1	$(\lvert\beta_1\rvert + \lvert\beta_2\rvert)/15 \le 1$, so $\lvert\beta_1\rvert + \lvert\beta_2\rvert \le 15$	diamond (Lasso)
0	$(\beta_1^2 + \beta_2^2)/225 \le 1$, so $\beta_1^2 + \beta_2^2 \le 225$	circle of radius 15 (Ridge)

r	Formula	Shape
0.5	half of each term	rounded diamond with corners kept

At $r = 0.5$ the point $(\beta_1, \beta_2) = (15, 0)$ gives

$$0.5 \times \frac{15}{15} + 0.5 \times \frac{225}{225}$$

$$= 0.5 + 0.5 = 1$$

so it sits exactly on the edge, on the axis. The point $(7.5, 7.5)$ gives

$$0.5 \times \frac{15}{15} + 0.5 \times \frac{112.5}{225}$$

$$= 0.5 + 0.25 = 0.75$$

so it is inside.

The picture follows the estimation picture of ESL Figure 3.11 and the shapes of Zou and Hastie (2005, Fig. 1), redrawn with our data.

5. Correlated features: the grouping effect

> **Key point:** When **features** (G-772) are strongly correlated, Lasso keeps one or two and drops the rest, with little regard for which. Elastic Net shares the weight among them and still drops the useless features.

Elastic Net is especially recommended when features are strongly correlated with each other: **multicollinearity** (G-1273; see no multicollinearity). Height and weight are a typical pair: when one rises, the other usually does too.

Figure 6 uses 200 **observations** (G-1374) (records, one row of the data table each) with six features. x_1, x_2 and x_3 are three almost identical copies of one signal, and the **target** (G-1949) (the output we predict) is 3 times that signal plus noise. The other three features are pure noise.

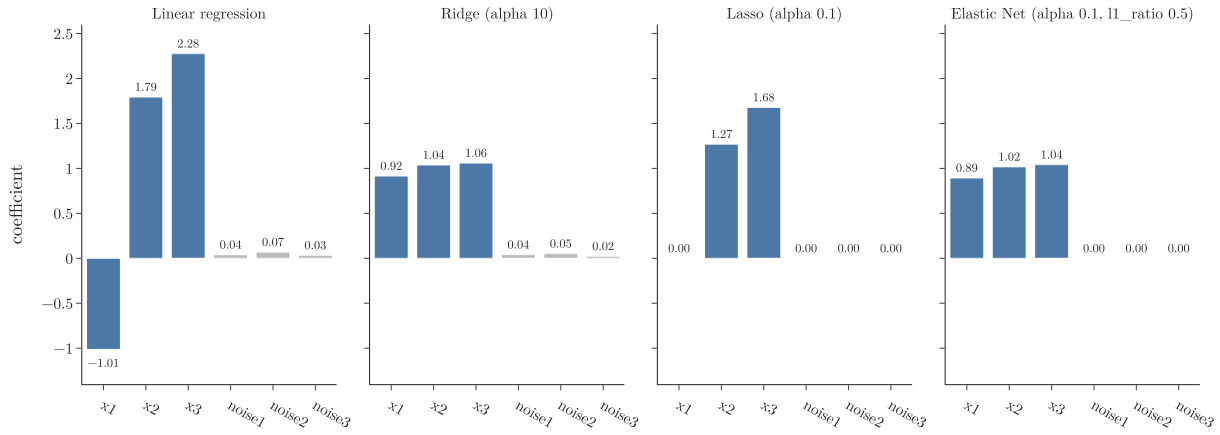


Figure 6: Coefficients of four models on three correlated inputs and three noise inputs

Model	x_1	x_2	x_3	Noise features
Linear regression	−1.01	1.79	2.28	about 0.05
Ridge (alpha 10)	0.92	1.04	1.06	about 0.04
Lasso (alpha 0.1)	0	1.27	1.68	0
Elastic Net (alpha 0.1, l1_ratio 0.5)	0.89	1.02	1.04	0

- **Linear regression** gives the three copies wild, unstable values; x_1 even gets a negative sign. Only their sum, about 3, is meaningful. With correlated features, a large coefficient on one copy can be cancelled by an opposite one on another (ESL §3.4.1).
- **Ridge** shares the weight evenly, about 1 each, but keeps small coefficients on the noise features.
- **Lasso** drops the noise, but also drops x_1, which is just as useful as the others. Which copy it drops is close to arbitrary: from a group of highly correlated features, Lasso tends to keep one and does not care which (Zou and Hastie 2005, §1). Here it kept two of the three.
- **Elastic Net** does both: it shares the weight evenly among the three copies, like Ridge, and sets the noise features to 0, like Lasso.

Figure 7 shows the step from one model to the next. The data and alpha (0.1) stay fixed, and only `l1_ratio` moves from 0 (the Ridge end) to 1 (Lasso). Watch the three grey noise bars first, then the bar of x_1.

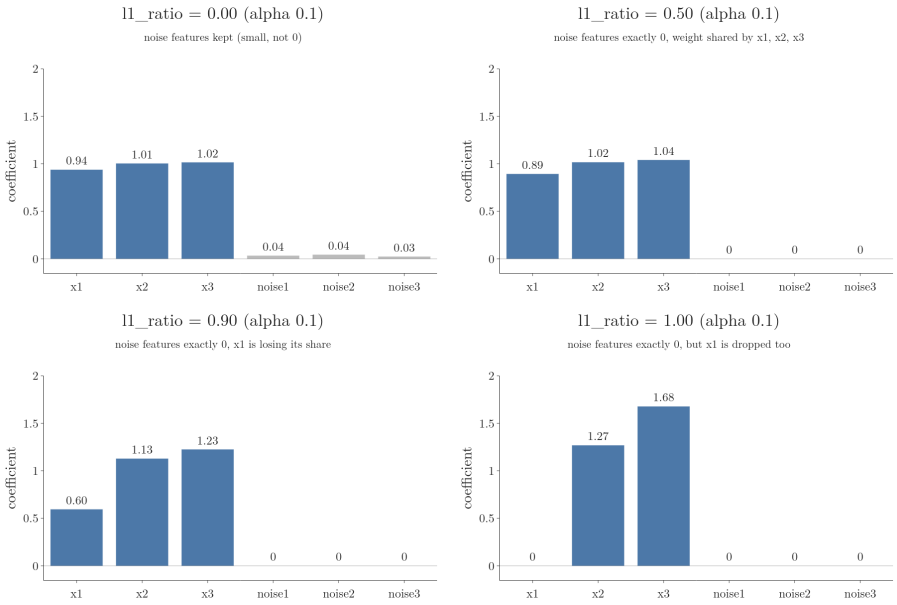


Figure 7: The six Elastic Net coefficients on the data of Figure 6 while l1_ratio grows from 0 to 1 at alpha 0.1. The noise coefficients are small but not 0 at the Ridge end and exactly 0 from l1_ratio 0.5 on. The three copies share the weight until the Lasso end, where x1 loses its share and is dropped

1. **l1_ratio 0 (the Ridge end):** the three copies share the weight (0.94, 1.01, 1.02), but the noise features keep small coefficients (about 0.04).
2. **l1_ratio 0.5:** the noise coefficients are exactly 0, and the copies still share the weight (0.89, 1.02, 1.04). This mix has both good properties.
3. **l1_ratio 0.9:** the sharing starts to break: x_1 is down to 0.60.
4. **l1_ratio 1 (Lasso):** x_1 is dropped (0, 1.27, 1.68).

This sharing of weight among correlated features is called the **grouping effect** (G-874). In short: Lasso tends to keep one feature of a correlated group and drop the rest, Ridge shrinks the group together, and Elastic Net keeps or removes the group together. For identical features, the Elastic Net penalty provably gives identical coefficients, while the Lasso penalty does not (Zou and Hastie 2005, §2.3).

6. Elastic Net on the diabetes data

> **Key point:** When the training set is small, all three penalties beat plain linear regression. Elastic Net, tuned by **cross-validation** (G-510), lands between Ridge and Lasso, and close to the better of the two.

A penalty is like a seatbelt. On a smooth road (plenty of data) it changes little. On a bumpy road (few **observations** (G-1374), correlated features) it keeps the coefficients from flying around. The diabetes data has bumps of the second kind: the features s1 and s2 have a correlation of 0.90.

We give each model only 80 training observations of the diabetes data and test on the

remaining 362. Every penalty strength (and, for Elastic Net, the l1_ratio) is chosen by 5-fold cross-validation on the training part only, never on the test set: the training part is cut into 5 folds, each fold in turn scores a model trained on the other 4, and the setting with the best average score wins. The table averages 40 random splits:

Test R^2 (see the R^2 score) is how much of the **target** (G-1949)'s variation the model explains on data it has not seen: 1 is perfect.

Model	Mean test R^2
Linear regression	0.412
Ridge (`RidgeCV`)	0.440
Lasso (`LassoCV`)	0.431
Elastic Net (`ElasticNetCV`)	0.437

Figure 8 sets these results beside the full-data results of the Extra below. Watch the red dots spread out with 80 training observations, and the blue diamonds bunch together with 353.

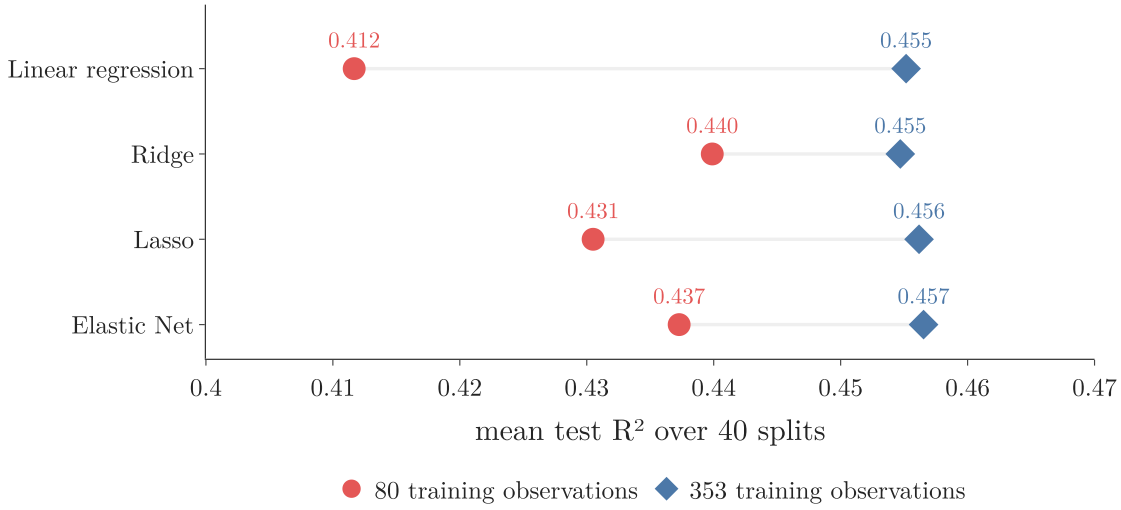


Figure 8: Mean test R^2 over 40 random splits of the diabetes data, with every penalty tuned by 5-fold cross-validation on the training part. Red: 80 training observations, where all penalties beat linear regression (0.412). Blue: 353 training observations, where all four tie (0.455 to 0.457).

- All three penalties beat plain linear regression. With few observations and correlated **features** (G-772), the least squares coefficients vary a lot from sample to sample, and shrinking them trades a small increase in bias for a large drop in variance (see the trade-off; ISL §6.2.1).
- Elastic Net beats Lasso. Lasso tends to keep one feature of a correlated group (Zou and Hastie 2005, §1): averaged over the splits, it sets 1.35 of the two correlated features s1

and s2 to zero, against 0.75 for Elastic Net, which shares the weight (the **grouping effect** (G-874) of Section 5).

- Ridge edges out Elastic Net slightly. Ridge does best when most features carry some signal (ISL §6.2.2), and most diabetes features do: even Lasso keeps about 7 of the 10 (it sets 3.3 to zero on average). Elastic Net does not know that in advance, but cross-validation steers it close.

Python: Tuning alpha and l1_ratio with cross-validation.

```
from sklearn.linear_model import ElasticNetCV

cv = ElasticNetCV(l1_ratio=[0.1, 0.5, 0.7, 0.9, 0.95, 0.99, 1],
                  alphas=np.logspace(-4, 1, 40), cv=5)
cv.fit(X_train, y_train)
cv.alpha_, cv.l1_ratio_            # the chosen settings
cv.score(X_test, y_test)           # R^2 on unseen data
```

Grid search and randomised search (later Notes) do the same tuning job for any model.

Extra: The condition matters. With the full training set (353 observations, test size 0.2), the four models tie: mean test R^2 0.455, 0.455, 0.456 and 0.457 over 40 splits. With enough observations, the least squares coefficients are already stable, so a penalty has little variance to remove (ISL ch. 6 introduction). Tuning the penalty against the test set itself would also flatter the scores; cross-validation on the training part avoids that.

Extra: `SGDRegressor(penalty="elasticnet", alpha=..., l1_ratio=...)` trains the same kind of model with stochastic gradient descent (how it works). scikit-learn recommends `SGDRegressor` for large datasets (more than 10,000 observations) and `ElasticNet` otherwise (scikit-learn user guide §1.5.2). `SGDRegressor` can also learn from data too large to fit in memory, because its `partial_fit` method trains on one small batch of rows at a time (scikit-learn user guide §10.1.1, out-of-core learning).

7. When to use which

Key point: Ridge when all **features** (G-772) matter, Lasso when only some do, Elastic Net when unsure or when features are strongly correlated.

Situation	Choice
Most features are useful	Ridge
Only some features are useful; want feature selection	Lasso
Many features, unclear which kind	Elastic Net
Strongly correlated features	Elastic Net (or Ridge)

The first two rows follow ISL §6.2.2; the last follows the **grouping effect** (G-874) above (Zou and Hastie 2005).

In practice, Elastic Net with l1_ratio tuned by **cross-validation** (G-510) covers all three cases: the search can land on Ridge-like or Lasso-like settings if those fit best.

8. Summary

- Elastic Net loss = squared error + L2 penalty + L1 penalty, so there is no need to choose between Ridge and Lasso before knowing which **features** (G-772) are useful.
- `alpha` sets the total strength; `l1_ratio` sets the L1 (Lasso) share: 0 is Ridge, 1 is Lasso; so one model covers both ends and everything in between.
- Elastic Net can still set coefficients to 0, because its penalty shape keeps the corners on the axes, but shares weight among correlated features (the **grouping effect** (G-874)), where Lasso keeps one copy and drops the others almost at random.
- Tune alpha and l1_ratio with **cross-validation** (G-510), for example with `ElasticNetCV`, so the search can land on Ridge-like or Lasso-like settings when those fit best.
- So Elastic Net is the safe choice when there are many features of unknown usefulness, or strongly correlated ones.

9. Sources

Built from

- CampusX, "ElasticNet Regression | Intuition and Code Example | Regularized Linear Models", YouTube, https://www.youtube.com/watch?v=2g2DBkFhTTY
- StatQuest with Josh Starmer, "Regularization Part 3: Elastic Net Regression", YouTube, https://www.youtube.com/watch?v=1dKRdX9bfIo

Other references

- **Zou and Hastie 2005:** Zou, H. and Hastie, T. "Regularization and Variable Selection via the Elastic Net." *Journal of the Royal Statistical Society B* 67(2), 301–320, 2005. Sections 1, 2.1 (Fig. 1) and 2.3.
- **ESL:** Hastie, T., Tibshirani, R. and Friedman, J. *The Elements of Statistical Learning*, 2nd ed. Springer, 2009. Section 3.4.1, p. 63; Section 3.4.3, Figure 3.11.
- **ISL:** James, G., Witten, D., Hastie, T. and Tibshirani, R. *An Introduction to Statistical Learning*, 2nd ed. Springer, 2021. Chapter 6 introduction, pp. 225–226; Section 6.2.1, p. 241; Section 6.2.2, pp. 246–247.
- **scikit-learn docs:** `sklearn.linear_model.ElasticNet`; user guide Section 1.5.2 (SGD regression) and Section 10.1.1 (*Scaling with instances using out-of-core learning*, incremental learning with `partial_fit`), scikit-learn 1.9.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Elastic Net regression (Elastic Net) (G-667)	Linear regression whose loss gets both the L2 (ridge) and the L1 (lasso) penalty, each with its own strength. It shrinks the coefficients and can set some to 0, so we need not choose between ridge and lasso in advance.
l1_ratio (G-1027)	The share of the total penalty given to the L1 (Lasso) part.
Grouping effect (G-874)	Elastic Net's tendency to give correlated inputs similar coefficients instead of keeping only one.
ElasticNetCV (G-669)	scikit-learn's Elastic Net that picks alpha and l1_ratio by cross-validation.
L2 regularisation (G-1029)	Another name for the squared-coefficient penalty used by Ridge.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
L1 regularisation (G-1026)	Another name for the absolute-value penalty used by Lasso.
Hyperparameter (G-910)	A setting of an algorithm chosen before training, such as a tree's `max_depth`.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Cross-validation (G-510)	Testing a model by training and testing it several times on different parts of the training data.
Feasible region (G-759)	All the points that obey every rule (constraint) of an optimisation problem; the answer must be one of them.
Multicollinearity (G-1273)	A relationship between input columns, so that one can be calculated (or largely predicted) from the others; it makes a linear model's coefficients unstable and hard to interpret.
Observation (G-1374)	One record of a dataset: one row of the data table.